Preprocessing Unstructured Data for LLM Applications

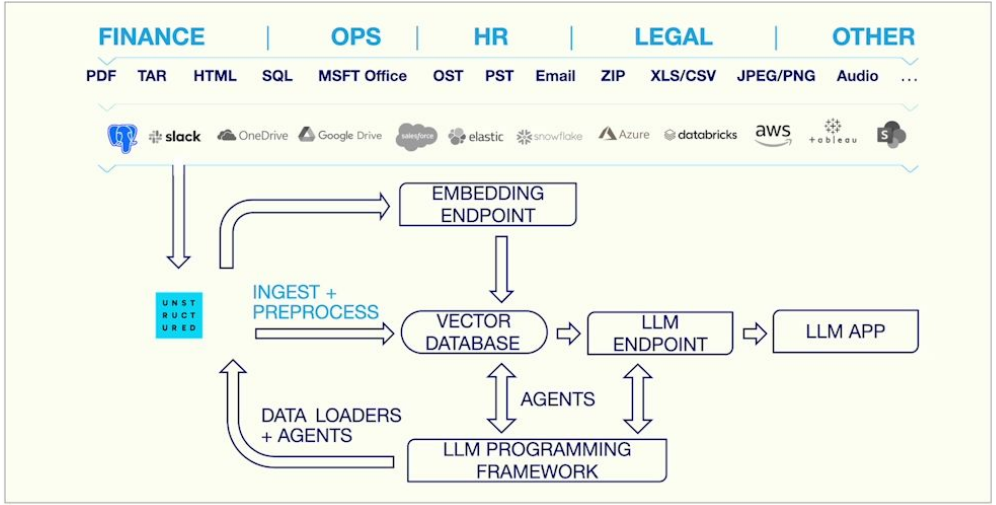
Data Preprocessing and LLMs

Retrieval Augmented Generation (RAG): A technique for grounding LLM responses on validated external information

Contextual Integration: RAG apps load context into a database, then retrieve content to insert into a prompt

Industry Application

For industry use cases, relevant content comes from diverse document types, such as PDFs, Word docs, email, and webpages



Preprocessing Outputs

Document Content: Text content from documents. Used for keyword or similarity search in RAG apps

Document Elements

The basic building blocks of a document. Useful for various RAG tasks, such as filtering and chunking

- Title
- Narrative text
- List item
- Table
- Image

Element Metadata

Additional information about element. Useful for filtering in hybrid search and for identifying the source of a response

- Filename
- Filetype
- Page number
- Section

Why is Data Preprocessing Hard?

Content Cues

Different document types have different cues for element types (visual, markdown).

Standardization Need

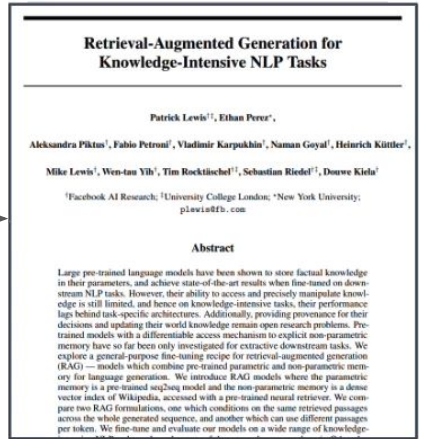
To process content from different document types, they need to be standardized.

Extraction Variability

Different documents format may require different extraction approaches (i.e., forms vs journal articles).

Metadata Insight

In many cases, extracting metadata requires an understanding of document structure



Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis[†‡], Ethan Perez[*],

Aleksandra Piktus[†], Fabio Petroni[†], Vladimir Karpukhin[†], Naman Goyal[†], Heinrich Küttler[†], Mike Lewis[†], Wen-tau Yih[†], Tim Rocktäschel[†‡], Sebastian Riedel[†‡], Douwe Kiela[†]

[†]Facebook AI Research; [‡]University College London; [*]New York University; plewis@fb.com

Abstract

Large pre-trained language models have been shown to store factual knowledge in their parameters, and achieve state-of-the-art results when fine-tuned on downstream NLP tasks. However, their ability to access and precisely manipulate knowledge is still limited, and hence on knowledge-intensive tasks, their performance lags behind task-specific architectures. Additionally, providing provenance for their decisions and updating their world knowledge remain open research problems. Pre-trained models with a differentiable access mechanism to explicit non-parametric memory have so far been only investigated for extractive downstream tasks. We explore a general-purpose fine-tuning recipe for retrieval-augmented generation (RAG) — models which combine pre-trained parametric and non-parametric memory for language generation. We introduce RAG models where the parametric memory is a pre-trained seq2seq model and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever. We compare two RAG formulations, one which conditions on the same retrieved passages across the whole generated sequence, and another which can use different passages per token. We fine-tune and evaluate our models on a wide range of knowledge-

Normalizing Diverse Documents

Format Diversity: Documents come in a variety of formats (PDF, Word, Markdown, etc)

Common Format: The first step in preprocessing is to convert raw documents into a common format that identifies common document elements like titles and narrative text

Normalizing Benefit: Normalized format allows any document to be processed in the same way, regardless of the source format

- Filtering out unwanted elements, like headers and footer
- Chunking document elements into sections

Reduced Processing Cost: The initial document processing steps is the most expensive part of the process

- Downstream tasks like chunking are inexpensive operations on normalized outputs
- Enables experiment with many different chunking techniques without reprocessing documents

Data Serialization

Serialization Benefits: Serialization allows the results of document processing to be used again later.

Advantages of JSON

- Structure is common and well understood
- Is a standard HTTP response
- Able to be used in multiple programming languages
- Can be converted to JSONL for streaming use cases

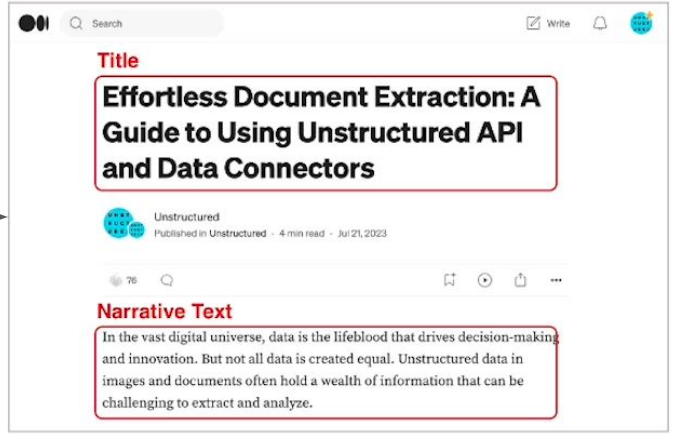

```
[
    {
        "element_id": "bff1fd0ec25e78f1224ad7309a1e79c4",
        "metadata": {
            "filename": "CoT.pdf",
            "filetype": "application/pdf",
            "languages": [
                "eng"
            ],
            "page_number": 1
        },
        "text": "B All Experimental Results",
        "type": "Title"
    },
    {
        "element_id": "ebf8dfb149bcbbd8c4b7f9a7046900a9",
        "metadata": {
            "filename": "CoT.pdf",
            "filetype": "application/pdf",
            "languages": [
                "eng"
            ],
            "page_number": 1,
            "parent_id": "bff1fd0ec25e78f1224ad7309a1e79c4"
        },
        "text": "This section contains tables for experimental results for varying models and model sizes, on all benchmarks, for standa
rd prompting vs. chain-of-thought prompting.",
        "type": "NarrativeText"
    }
]
```

HTML Page

LLM Relevance: Integrating fresh internet data into LLMs to main there currentness and relevance

HTML Understanding: Essential for structuring web content, using elements like <h1> for titles and <p> for text

Data Extraction and Categorization: Analyzing HTML elements to extract and organize web content into structured formats for relevant information distillation

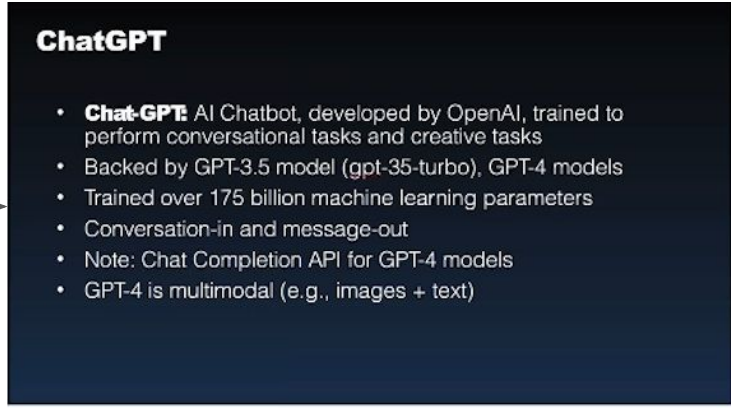


MS Powerpoint

Professional Use: Widely used in consulting to present ideas, strategies, and results, crucial for enhancing LLM's knowledge base

Extraction Process: Involves parsing elements like bulleted paragraphs, slide notes, shapes, and tables in Powerpoint slides

Tool Utilization: Python libraries, such as 'pptx', navigate and extract textual or visual information from slides



PDF with Complex Layout and Tables

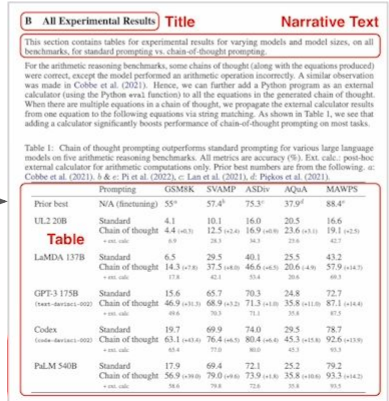
PDF Characteristics

- Maintains consistent formatting across devices
- Complex structures with diverse layouts, including text, images, and tables in non-sequential order

Extraction Tools and Libraries

- Facilitate text and element extraction
- Preserve context and layout, distinguishing between main text, headers, footers, and side notes.

Advanced Capabilities: Use OCR and Transformer-based technology



B All Experimental Results

Title

Narrative Text

This section contains tables for experimental results for varying models and model sizes, on all benchmarks, for standard prompting vs. chain-of-thought prompting.

For the arithmetic reasoning benchmarks, some chains of thought (along with the equations produced) were correct, except the model performed an arithmetic operation incorrectly. A similar observation was made in Cobbe et al. (2021). Hence, we can further add a Python program as an external calculator (using the Python `eval` function) to all the equations in the generated chain of thought. When there are multiple equations in a chain of thought, we propagate the external calculator results from one equation to the following equations via string matching. As shown in Table 1, we see that adding a calculator significantly boosts performance of chain-of-thought prompting on most tasks.

Table 1: Chain of thought prompting outperforms standard prompting for various large language models on five arithmetic reasoning benchmarks. All metrics are accuracy (%). Ext. calc.: post-hoc external calculator for arithmetic computations only. Prior best numbers are from the following. *a*: Cobbe et al. (2021). *b* & *c*: Pi et al. (2022), *c*: Lan et al. (2021), *d*: Piękos et al. (2021).

Table

	Prompting	GSM8K	SVAMP	ASDiv	AQuA	MAWPS
Prior best	N/A (finetuning)	55[a]	57.4[b]	75.3[c]	37.9[d]	88.4[c]
UL2 20B	Standard	4.1	10.1	16.0	20.5	16.6
	Chain of thought	4.4 (+0.3)	12.5 (+2.4)	16.9 (+0.9)	23.6 (+3.1)	19.1 (+2.5)
	+ ext. calc	6.9	28.3	34.3	23.6	42.7
LaMDA 137B	Standard	6.5	29.5	40.1	25.5	43.2
	Chain of thought	14.3 (+7.8)	37.5 (+8.0)	46.6 (+6.5)	20.6 (-4.9)	57.9 (+14.7)
	+ ext. calc	17.8	42.1	53.4	20.6	60.3
GPT-3 175B	Standard	15.6	65.7	70.3	24.8	72.7
(text-davinci-002)	Chain of thought	46.9 (+31.3)	68.9 (+3.2)	71.3 (+1.0)	35.8 (+11.0)	87.1 (+14.4)
	+ ext. calc	49.6	70.3	71.1	35.8	87.5
Codex	Standard	19.7	69.9	74.0	29.5	78.7
(code-davinci-002)	Chain of thought	63.1 (+43.4)	76.4 (+6.5)	80.4 (+6.4)	45.3 (+15.8)	92.6 (+13.9)
	+ ext. calc	65.4	77.0	80.0	45.3	93.3
PaLM 540B	Standard	17.9	69.4	72.1	25.2	79.2
	Chain of thought	56.9 (+39.0)	79.0 (+9.6)	73.9 (+1.8)	35.8 (+10.6)	93.3 (+14.2)
	+ ext. calc	58.6	79.8	72.6	35.8	93.5

What is Metadata?

Document Details: Metadata provides additional information about content extract from source documents.

Source Identification: One type of metadata is information about document itself, such as filename, source URL, or file type

Structural Metadata: Metadata can be constructed from the structure of the document.

Examples: Element type, hierarchy information, and section information.

Search Enhancement: In RAG systems, metadata provides filtering options for hybrid search

Metadata Example


```
{
  "type": "Title",
  "element_id": "0c6fff519d8892b1783ead469b230320",
  "text": "The Project Gutenberg eBook of Winter Sports in \ nSwitzerland, by E. F. Benson",
  "metadata": {
   "page_number": 1,
   "languages": [
    "eng"
   ],
   "filename": "winter-sports.epub",
   "filetype": "application/epub"
  }
}
```

Semantic Search for LLMs

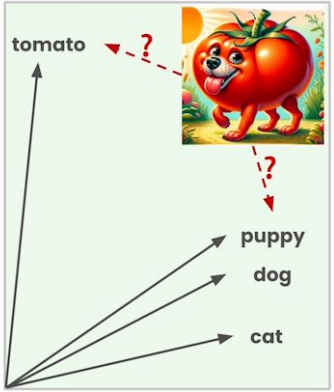
Semantic Search with Vector Databases

- **Goal:** Given an input text, find similarity similar content from a corpus of documents for use in prompt techniques
- **Embedding:** Convert a text to vectors that can be compared through a similarity function, such as cosine similarity.
- **Vector Database:** A database optimized for performing similarity search.
- **Prompt Templating:** Insert relevant content into a template to generate a prompt for the LLM

Load: Insert the vectors into the database, along with the source documents or a pointer to the source document

Query Embed: Embed the input for the similarity search

Compare and Retrieve: Compare the query embedding to documents in the vectorDB; retrieve the k most similar documents



Hybrid Search

Semantic Similarity Search Challenges

- **Too Many Matches:** In cases with a large number of documents, too many semantically similar matches
- **Most Recent Information:** Users may want the most recent information, not just the most semantically similar
- **Loss of Important Information:** Loss of important information within the document that's relevant to the search, such as section information.

Hybrid Strategy: Hybrid search is a search strategy that combines semantic search with other information retrieval techniques, such as filtering and keyword search.

Filtering Options: Metadata from documents provides filtering options for hybrid search.

Example: Most Recent Information

February **12**, 2024

Chiefs defeat 49ers 25-22 in OT thriller in Super Bowl 2024. Here are highlights from the big game.

The Kansas City Chiefs overcame a 10-point deficit to become the first back-to-back Super Bowl champion in two decades as they defeated the San Francisco 49ers 25-22 in overtime in Super Bowl LVIII in Las Vegas.

In a back-and-forth affair, the two teams were tied at 19 at the end of regulation. Jake Moody hit a 27-yard field goal in overtime to give the Niners a 22-19 lead. But Kansas City quarterback Patrick Mahomes, as he has so many times before, drove the Chiefs down the field, and with 13 seconds left in overtime.

Source: *CBS Sports*

February **11**, 2024

Who is favored to win the 2024 Super Bowl, and which team is the underdog?

Despite their status as defending champions, the Kansas City Chiefs are slight underdogs when they face the San Francisco 49ers in the 2024 Super Bowl in Las Vegas today.

The Niners first opened as 1.5-point favorites, according to most sportsbooks. The line has since shifted even more in their favor, with San Francisco settling in as consensus 2-point favorites by Saturday afternoon.

Source: *CBS Sports*

Chunking

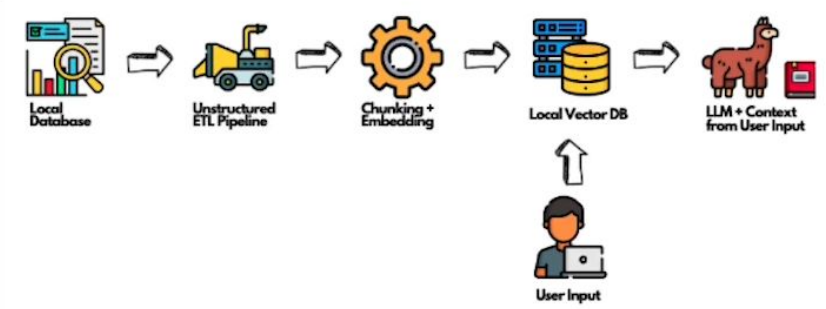
Chunking Necessity: Vector databases need documents split into chunks for retrieval and prompt generation

Query Result Variability: The same query will return different content depending on how the document is chunked

Even Size Chunks: The easiest way is to split the document into roughly even size chunks. This can result in similar content getting split across chunks

Chunking by Atomic Elements: By identifying atomic elements, you can chunk by combining elements rather than splitting raw text

- Result in more coherent chunks
- **Example**: combining content under the same section header into the same chunk.



Chunking From Elements

Partitioning: First breakdown the document down into atomic elements

Combine Elements into Chunks: Add one document element to the chunk. Continue adding document elements to the chunk until you've reached a character or token threshold

Apply Break Conditions: Apply a condition for starting a new chunk, such as when we reach a new title element (indicating a new section), when element metadata changes (indicating a new page or section), or when content similarity exceeds a threshold.

Basic combinative chunking applies no break conditions

Combine Smaller Chunks: Optionally, combine small chunks so chunks are sufficiently large for effective semantic searches

Coherent Chunks: Keeps content from the same document element together, resulting in more coherent chunks.

Structured Chunks: Allows the chunking method to take advantage of the document structure, which is not the case for traditional chunking techniques that split on token or character count.

Rapid Experimentation: Building chunks from document elements allows for rapid experimentation (partitioning is expensive, chunking is fast).

Example

Character Splitting

3.1 Open-domain Question Answering

Open-domain question answering (QA) is an important real-world application and common testbed for knowledge-intensive tasks [20]. We treat questions and answers as input-output text pairs (x, y) and train RAG by directly minimizing the negative log-likelihood of answers. We compare RAG to the popular extractive QA paradigm [5, 7, 31, 26], where answers are extracted spans from retrieved documents, relying primarily on non-parametric knowledge. We also compare to "Closed-Book QA" approaches [52], which, like RAG, generate answers, but which do not exploit retrieval, instead relying purely on parametric knowledge. We consider four popular open-domain QA datasets: Natural Questions (NQ) [29], TriviaQA (TQA) [24]. WebQuestions (WQ) [3] and CuratedTrec (CT) [2]. As CT and WQ are small, we follow DPR [26] by initializing CT and WQ models with our NQ RAG model. We use the same train/dev/test splits as prior work [31, 26] and report Exact Match (EM) scores. For TQA, to compare with T5 [52], we also evaluate on the TQA Wiki test set.

3.2 Abstractive Question Answering

RAG models can go beyond simple extractive QA and answer questions with free-form, abstractive text generation. To test RAG's natural language generation (NLG) in a knowledge-intensive setting, we use the MSMARCO NLG task v2.1 [43]. The task consists of questions, ten gold passages retrieved from a search engine for each question, and a full sentence answer annotated from the retrieved passages. We do not use the supplied passages, only the questions and answers, to treat

Chunking with Document Elements

3.1 Open-domain Question Answering

Open-domain question answering (QA) is an important real-world application and common testbed for knowledge-intensive tasks [20]. We treat questions and answers as input-output text pairs (x, y) and train RAG by directly minimizing the negative log-likelihood of answers. We compare RAG to the popular extractive QA paradigm [5, 7, 31, 26], where answers are extracted spans from retrieved documents, relying primarily on non-parametric knowledge. We also compare to "Closed-Book QA" approaches [52], which, like RAG, generate answers, but which do not exploit retrieval, instead relying purely on parametric knowledge. We consider four popular open-domain QA datasets: Natural Questions (NQ) [29], TriviaQA (TQA) [24]. WebQuestions (WQ) [3] and CuratedTrec (CT) [2]. As CT and WQ are small, we follow DPR [26] by initializing CT and WQ models with our NQ RAG model. We use the same train/dev/test splits as prior work [31, 26] and report Exact Match (EM) scores. For TQA, to compare with T5 [52], we also evaluate on the TQA Wiki test set.

3.2 Abstractive Question Answering

RAG models can go beyond simple extractive QA and answer questions with free-form, abstractive text generation. To test RAG's natural language generation (NLG) in a knowledge-intensive setting, we use the MSMARCO NLG task v2.1 [43]. The task consists of questions, ten gold passages retrieved from a search engine for each question, and a full sentence answer annotated from the retrieved passages. We do not use the supplied passages, only the questions and answers, to treat

Document Image Analysis

Processing with Rules-based Parsers: Many document types, such as HTML, Word Docs, and Markdown include formatting information. These documents can be preprocessed with rules-based parsers

Visual Information: For other documents, such as PDFs, and images, the formatting information is visual

Document Image Analysis (DIA): allows us to extract formatting information and text from the raw image of a document

Document Image Analysis Methods

Document Layout Detection: uses an object detection to model to draw and label bounding boxes around layout elements on a document image.

Vision Transformers (ViT): models take a document image as input and produce a text representation of a structured output (like JSON) as output.

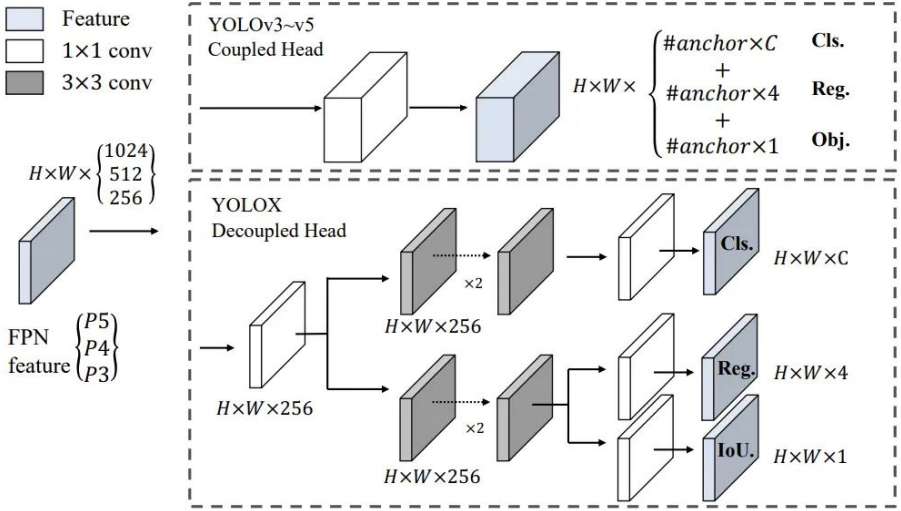
Note: ViTs can optionally include a text prompt.

Document Layout Detection

Vision Detection: Identify and classify bounding box using a computer vision model, such as YOLOX or Detectron2.

Text Extraction: Extract text from the bounding box using OCR when necessary

Direct Extraction: For some documents like PDFs, text can be extracted directly from the document without OCR.



Document Layout Detection

B All Experimental Results

Title

Narrative Text

This section contains tables for experimental results for varying models and model sizes, on all benchmarks, for standard prompting vs. chain-of-thought prompting.

For the arithmetic reasoning benchmarks, some chains of thought (along with the equations produced) were correct, except the model performed an arithmetic operation incorrectly. A similar observation was made in Cobbe et al. (2021). Hence, we can further add a Python program as an external calculator (using the Python `eval` function) to all the equations in the generated chain of thought. When there are multiple equations in a chain of thought, we propagate the external calculator results from one equation to the following equations via string matching. As shown in Table 1, we see that adding a calculator significantly boosts performance of chain-of-thought prompting on most tasks.

Table 1: Chain of thought prompting outperforms standard prompting for various large language models on five arithmetic reasoning benchmarks. All metrics are accuracy (%). Ext. calc.: post-hoc external calculator for arithmetic computations only. Prior best numbers are from the following. *a*: Cobbe et al. (2021). *b* & *e*: Pi et al. (2022), *c*: Lan et al. (2021), *d*: Piękos et al. (2021).

Table

	Prompting	GSM8K	SVAMP	ASDiv	AQuA	MAWPS
Prior best	N/A (finetuning)	55[a]	57.4[b]	75.3[c]	37.9[d]	88.4[e]
UL2 20B	Standard	4.1	10.1	16.0	20.5	16.6
	Chain of thought	4.4 (+0.3)	12.5 (+2.4)	16.9 (+0.9)	23.6 (+3.1)	19.1 (+2.5)
	+ ext. calc	6.9	28.3	34.3	23.6	42.7
LaMDA 137B	Standard	6.5	29.5	40.1	25.5	43.2
	Chain of thought	14.3 (+7.8)	37.5 (+8.0)	46.6 (+6.5)	20.6 (-4.9)	57.9 (+14.7)
	+ ext. calc	17.8	42.1	53.4	20.6	69.3
GPT-3 175B (text-davinci-002)	Standard	15.6	65.7	70.3	24.8	72.7
	Chain of thought	46.9 (+31.3)	68.9 (+3.2)	71.3 (+1.0)	35.8 (+11.0)	87.1 (+14.4)
	+ ext. calc	49.6	70.3	71.1	35.8	87.5
Codex (code-davinci-002)	Standard	19.7	69.9	74.0	29.5	78.7
	Chain of thought	63.1 (+43.4)	76.4 (+6.5)	80.4 (+6.4)	45.3 (+15.8)	92.6 (+13.9)
	+ ext. calc	65.4	77.0	80.0	45.3	93.3
PaLM 540B	Standard	17.9	69.4	72.1	25.2	79.2
	Chain of thought	56.9 (+39.0)	79.0 (+9.6)	73.9 (+1.8)	35.8 (+10.6)	93.3 (+14.2)
	+ ext. calc	58.6	79.8	72.6	35.8	93.5

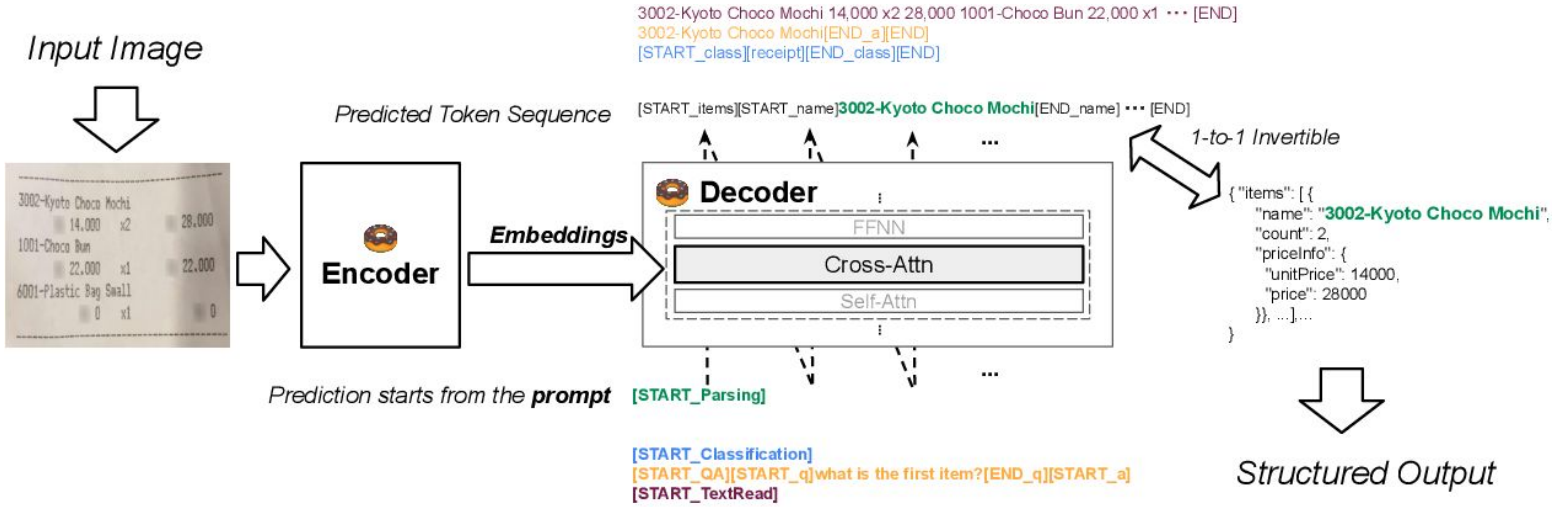
Vision Transformers

Visual Translation: Input image is passed to the encoder and the decoder produces text output

DONUT Architecture: Document Understanding Transformer (DONUT) is a common architecture

Direct Conversion: OCR is not required - the image input is converted directly to text.

Structured Training: Can train the model to output a valid JSON string with the structured document output.



Vision Transformers

Course Overview

- Lesson 0: Intro to Data Preprocessing for LLM Applications
- Lesson 1: Normalizing Content from Diverse Document Types
- Lesson 2: Metadata Extraction for RAG Systems
- Lesson 3: Preprocessing PDFs and images with Document Image Analysis
- Lesson 4: Extracting Tables from Documents
- Lesson 5: Content Aware chunking

Vision Transformer


```
[
  {
    "type": "Title",
    "text": "Course Overview"
  },
  {
    "type": "ListItem",
    "text": "Lesson 0: Intro to Data Preprocessing for LLM
Applications"
  },
  {
    "type": "ListItem",
    "text": "Lesson 1: Normalizing Content from Diverse Doc
ument Types"
  },
  {
    "type": "ListItem",
    "text": "Lesson 2: Metadata Extraction for RAG Systems"
  },
  {
    "type": "ListItem",
    "text": "Lesson 3: Preprocessing PDFs and images with D
ocument Image Analysis"
  },
  {
    "type": "ListItem",
    "text": "Lesson 4: Extracting Tables from Documents"
  },
  {
    "type": "ListItem",
    "text": "Lesson 5: Content Aware chunking"
  }
]
```

Advantages and Disadvantages

Document Layout Models

Advantages

- Have a fixed set of element types
- Get bounding box information

Disadvantages

- Requires two model calls (the object detection and OCR models)
- Less flexible

Vision Transformers

Advantages

- More flexible for non-standard document types like forms
- More adaptable to new ontologies

Disadvantages

- Model is generative, potentially prone to hallucination or repetition.
- Computationally expensive

Table Extraction

Text Analysis: Most RAG use cases focus on text content within documents.

Structured Data: Some industries (i.e., finance, insurance) deal heavily with structured data embedded in unstructured documents

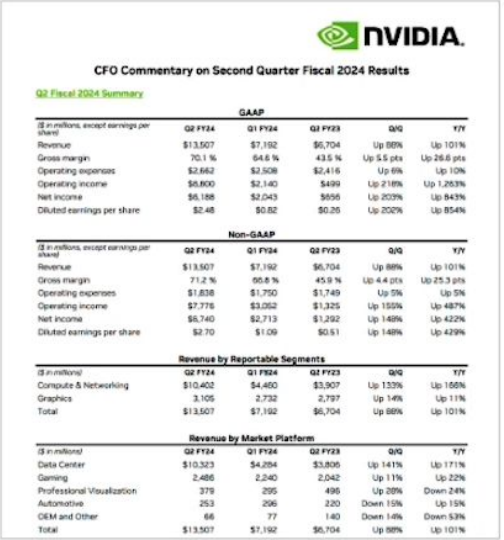
Table Extraction: To support use cases such as question answering over tables, it is useful to extract tables from documents.

Inherent Structures: Some documents (HTML, Word Docs) contain table structure information.

Inference Required: For other documents (PDF, images) table information needs to be inferred.

Techniques: Table Transformers, Vision Transformers, OCR Postprocessing.

HTML Output: Want to extract tables in HTML format to preserve structure.



NVIDIA.

CFO Commentary on Second Quarter Fiscal 2024 Results

Q2 Fiscal 2024 Summary

GAAP

($ in millions, except earnings per share)	Q2 FY24	Q1 FY24	Q2 FY23	Q/Q	Y/Y
Revenue	$13,507	$7,192	$6,704	Up 88%	Up 101%
Gross margin	70.1 %	64.6 %	43.5 %	Up 5.5 pts	Up 26.6 pts
Operating expenses	$2,662	$2,508	$2,416	Up 6%	Up 10%
Operating income	$6,800	$2,140	$499	Up 218%	Up 1,263%
Net income	$6,188	$2,043	$656	Up 203%	Up 843%
Diluted earnings per share	$2.48	$0.82	$0.26	Up 202%	Up 854%

Non-GAAP

($ in millions, except earnings per share)	Q2 FY24	Q1 FY24	Q2 FY23	Q/Q	Y/Y
Revenue	$13,507	$7,192	$6,704	Up 88%	Up 101%
Gross margin	71.2 %	66.8 %	45.9 %	Up 4.4 pts	Up 25.3 pts
Operating expenses	$1,838	$1,750	$1,749	Up 5%	Up 5%
Operating income	$7,776	$3,062	$1,325	Up 155%	Up 487%
Net income	$6,740	$2,713	$1,292	Up 148%	Up 422%
Diluted earnings per share	$2.70	$1.09	$0.51	Up 148%	Up 429%

Revenue by Reportable Segments

($ in millions)	Q2 FY24	Q1 FY24	Q2 FY23	Q/Q	Y/Y
Compute & Networking	$10,402	$4,460	$3,907	Up 133%	Up 166%
Graphics	3,105	2,732	2,797	Up 14%	Up 11%
Total	$13,507	$7,192	$6,704	Up 88%	Up 101%

Revenue by Market Platform

($ in millions)	Q2 FY24	Q1 FY24	Q2 FY23	Q/Q	Y/Y
Data Center	$10,323	$4,284	$3,806	Up 141%	Up 171%
Gaming	2,486	2,240	2,042	Up 11%	Up 22%
Professional Visualization	379	295	496	Up 28%	Down 24%
Automotive	253	296	220	Down 15%	Up 15%
OEM and Other	66	77	140	Down 14%	Down 53%
Total	$13,507	$7,192	$6,704	Up 88%	Up 101%

Table Transformers

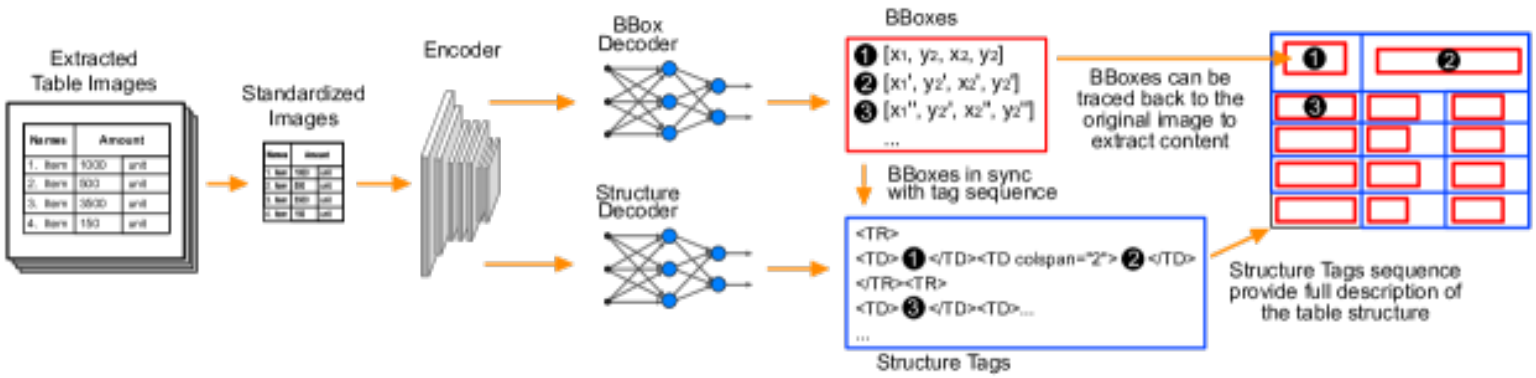
Table transformer is a model that identifies bounding boxes for table cells and converts the output to HTML

Two Steps:

1. First identify tables using the document layout model
2. Then run the table through the table transformer

Advantages: can trace cells back to the original bounding boxes

Disadvantages: multiple expensive model calls



Vision Transformers

Use the vision transformer models, but target HTML as output

Advantages: allows for prompting; more flexible; one model call

Disadvantages: generative and prone to hallucination; no bounding boxes

Company/Index	April 2013	April 2014	April 2015	April 2016	April 2017	April 2018
Medtronic, Inc. / Medtronic plc	$ 100.00	$ 128.10	$ 173.85	$ 180.96	$ 194.03	$ 194.16
S&P 500 Index	100.00	120.27	139.48	139.05	163.96	187.24
S&P 500 Health Care Equipment Index	100.00	119.09	156.85	166.19	194.71	234.34

```
<table>
        <thead>
                <tr><td> Company/Index</td><td> April 2013</td><td> April 2014</td><td> April
        2015</td><td> April 2016</td><td> April 2017</td><td> April 2018</td></tr>
        </thead>
        <tbody>
                <tr><td> Medtronic, Inc. / Medtronic plc</td><td> $100.00</td><td> $128.10</td><td>
        $173.85</td><td> $180.96</td><td> $194.03</td><td> $194.16</td></tr>
                <tr><td> S&P 500 Index</td><td> 100.00</td><td> 120.27</td><td> 139.48</td><td>
        139.05</td><td> 163.96</td><td> 187.24</td></tr>
                <tr><td> S&P 500 Health Care Equipment Index</td><td> 100.00</td><td>
        119.09</td><td> 156.85</td><td> 166.19</td><td> 194.71</td><td> 234.34</td></tr>
        </tbody>
</table>
```

OCR Post Processing

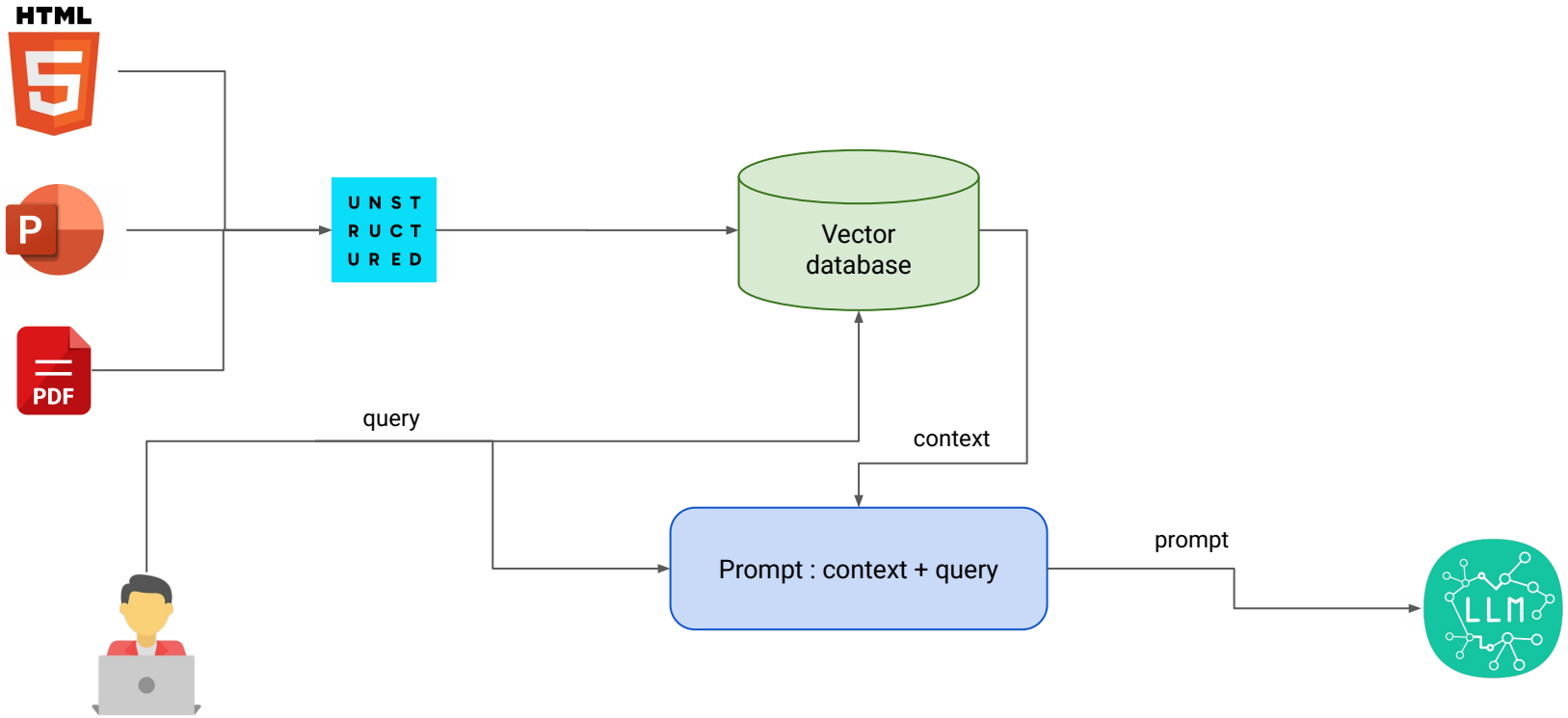
OCR the table, then build the table structure based on patterns in the OCR output

Advantages: fast; accurate for well behaved tables

Disadvantages: requires statistical or rules based parsing; less flexible than other approaches; no link back to bounding boxes in images

Item	Change	Section/page
1	" [Part 1: Administrative specifications]" removed	Title page, p.1
2	Deliverable name " 3 in depth policy reports to support R&I priority setting " replaced by " At least 2 in depth policy reports to support R & I priority settings " and Deadline "M9 - M21 - M33" replaced by "M16 - M33"	Section 1.4.6, Table 1 - Deliverables, p.20
3	" INEA estimates that tenderers should dedicate 15 to 20% of the contract value to works serving the BRIDGE initiative (Task 5, Task 6, Task 7 and BRIDGE-related activities in Task 8 and Task 9) and the remaining share to works serving the ETIP SNET. replaced by INEA estimates that tenderers should dedicate 15 to 20% of the contract value to works serving the BRIDGE initiative (Task 5, Task 6, and BRIDGE-related activities in Task 7, Task 8 and Task 9) and the remaining share to works serving the ETIP SNET.	Section 1.7, p.21
4	"This criterion applies to a staff member of the tenderer appointed as project manager." replaced by "This criterion applies to a staff member of the tenderer appointed as lead editor."	Section 3.2.3.2, Criterion T8, p.34

Build your own RAG Bot



THANK YOU